

Utilización de APIs Tridimensionales

Práctica 2b: Rotating triangles (OpenGL 4)

En esta práctica, a partir de la plantilla realizada en clase, vamos a dibujar con OpenGL 4 una serie de triángulos en pantalla. Partiendo de lo implementado en la práctica anterior, añadiremos nuevas clases para poder compilar Shaders y poder usar BufferObjects para almacenar los datos de las mallas en GPU.

Las clases que se muestran a continuación son “sugerencias”, se deja al alumno libertad de implementación. En cualquier caso, se usará un esquema similar al mostrado para las próximas prácticas, por lo que será responsabilidad del alumno poder adaptar su código a lo pedido.

API básica a implementar

Partiendo de las clases creadas anteriormente, añadiremos nuevas clases y modificaremos algunas para poder usar Shaders para dibujado de triángulos. A continuación se detallan las nuevas clases:

Clase abstracta Program

Esta clase abstracta “pura” representa la interfaz de un generador/intérprete de código que se le pasará a nuestro programa para poder dibujar en pantalla usando Hardware programable. En concreto, lo usaremos para dar una interfaz de carga y compilación de programas de tipo “Shader” de GLSL. Tendrá la siguiente interfaz:

Enumeradores de tipos de programa (de momento 2)

```
typedef enum class programTypes_e {  
    vertex = 0, fragment = 1  
}programTypes_e;
```

Atributos de clase:

- **programTypes_e type**: Tipo de programa cargado (vértice o fragmento)
- **std::string fileName**: Nombre del archivo cargado
- **unsigned int idProgram**: Identificador único de programa. Lo usaremos para guardar el identificador creado por OpenGL;

Métodos de clase:

- **Program(std::string fileName)**: Constructor de clase, inicializa la variable de nombre. Usaremos la extensión del archivo para seleccionar el tipo de programa que usaremos.
- **virtual void readFile()**: Método virtual puro de carga de código de ficheros.
- **virtual void compile()**: Método virtual puro que representa la ejecución de compilación de programa
- **virtual void checkErrors()**. Método que imprimirá por pantalla cualquier error encontrado durante la compilación o carga del archivo.

Clase abstracta RenderProgram

Esta es una clase abstracta “pura” que representará un programa encargado de realizar las tareas de renderizado en un HW especial (GPU en nuestro caso, aunque podría ser cualquier otro tipo de renderizador). Recibirá una lista de código fuente para tratar vértices y fragmentos, lo compilará y creará un programa para poder dibujar más adelante listas de vértices y triángulos.

Atributos privados:

- **std::vector<Program*> shaders:** Vector con los programas cargados para el dibujado (shaders de vértices y fragmentos en nuestro caso)
- **std::map<std::string, unsigned int> varList:** Mapa con la lista de variables activas encontrada en los programas cargados anteriormente

Métodos virtuales puros:

- **virtual void addProgram(std::string fileName):** Añade un programa identificado por su nombre de archivo para compilarlo
- **virtual void linkProgram():** Una vez añadidos todos los programas, compila y linka para generar el programa final
- **virtual void use():** Activa el uso de este programa
- **virtual void checkLinkerErrors():** Muestra por terminal los errores encontrados durante la compilación y linkado

Métodos públicos de acceso/set de variables del programa: Setean el valor de una variable a partir de su nombre

- **virtual void setVertexAttrib(std::string name, GLsizei stride, void* offset, GLint count, GLenum type)=0;**
- **virtual void setInt(std::string name, int val)=0;**
- **virtual void setFloat(std::string name, float val) = 0;**
- **virtual void setVec3(std::string name, const glm::vec3& vec) = 0;**
- **virtual void setVec4(std::string name, const glm::vec4& vec) = 0;**
- **virtual void setMatrix(std::string name, const glm::mat4& matrix) = 0;**

Clase GLSLShader

Esta clase implementará una interfaz de tipo Program basado en shaders de tipo GLSL. Debe proporcionar las implementaciones a los métodos anteriores. Se deja libertad al alumno para añadir todos los métodos necesarios que considere para poder compilar los shaders y generar un programa final de GPU. A continuación se dan algunas funciones de ayuda:

Método para averiguar si ha habido errores al compilar un shader:

```
void GLSLShader::checkErrors() {
```

```

GLint fragment_compiled;
glGetShaderiv(idProgram, GL_COMPILE_STATUS, &fragment_compiled);
if (fragment_compiled != GL_TRUE)
{
    GLsizei log_length = 0;
    GLchar message[1024];
    glGetShaderInfoLog(idProgram, 1024, &log_length, message);
    std::cout << "ERROR " << fileName << "\n" << message << "\n\n";
}
}

```

Método para leer un archivo de texto:

```

void GLSLShader::readFile() {
    std::ifstream f(fileName);
    if (f.is_open()) {
        code=std::string(std::istreambuf_iterator<char>(f), {});
        f.close();
    }
    else {
        std::cout << "ERROR: FICHERO NO ENCONTRADO " <<
            __FILE__ << ":" << __LINE__ << " " << fileName << "\n";
    }
}

```

Clase GLSLProgram

Clase que implementa una interfaz de tipo RenderProgram basada en Shaders de GLSL. Debe implementar todos los métodos virtuales de la clase RenderProgram, y se pueden añadir los métodos que se consideren necesarios para facilitar su uso.

A continuación se dan algunas ayudas para su implementación:

Método para interrogar a un shader y averiguar todas las variables de tipo uniform y attribute que pueden ser accedidas, y guardarlas en el mapa “varList”:

```

void GLSLProgram::readVarList() {

    int numAttributes = 0;
    int numUniforms = 0;
    glGetProgramiv(idRenderProgram, GL_ACTIVE_ATTRIBUTES, &numAttributes);
    for (int i = 0; i < numAttributes; i++)
    {
        char varName[100];
        int bufSize = 100, length = 0, size = 0;
        GLenum type = -1;
        glGetActiveAttrib(idRenderProgram, (GLuint)i, bufSize, &length, &size, &type,
varName);
        varList[std::string(varName)] = glGetAttribLocation(idRenderProgram, varName);
    }

    glGetProgramiv(idRenderProgram, GL_ACTIVE_UNIFORMS, &numUniforms);
    for (int i = 0; i < numUniforms; i++)
    {
        char varName[100];
        int bufSize = 100, length = 0, size = 0;
        GLenum type = -1;
        glGetActiveUniform(idRenderProgram, (GLuint)i, bufSize, &length, &size, &type,
varName);
        varList[std::string(varName)] = glGetUniformLocation(idRenderProgram, varName);
    }
}

```

```
}
```

Método para obtener el "Location" de una variable de GLSL, y que además muestra un error si no se ha encontrado (USADLO)

```
unsigned int GLSLProgram::getVarLocation(std::string varName) {
    if (varList.find(varName) != varList.end())
        return varList[varName];
    else {
        std::cout << "ERROR: variable " << varName <<
                    " no encontrada en shader\n";
        return -1;
    }
}
```

Clase abstracta Material

Los objetos 3d están formados por mallas 3D. Ahora las mallas tendrán asociada una clase que permitirá definir cómo se dibuja, colorea, textura, etc.. cada una de sus mallas. Los Materiales guardarán información sobre los programas (shaders) que se usarán para el dibujado de mallas. La clase básica Material ahora mismo tiene pocos métodos, más adelante crecerá para tener más información del tipo de material:

Atributos privados:

- **RenderProgram* program** :Puntero a una implementación de programas de renderizado (en nuestro caso, shaders). Deberá ser inicializado por el constructor de la clase que implemente un Material

Métodos virtuales puros:

- **loadPrograms**: Método virtual que recibirá una lista de nombres de archivos de programas (al menos uno de vértices y uno de fragmentos). Deberá pasárselos al RenderProgram para que los compile y linke adecuadamente.
- **prepare()**: Método virtual que inicializará adecuadamente las variables del RenderProgram a partir de los objetos

Clase GLSLMaterial

Implementa una clase Material con un GLSLProgram como RenderProgram. Implementará los métodos virtuales comentados anteriormente. Se pueden añadir los métodos privados/públicos que considere el alumno.

Modificaciones en clase Mesh3D

Se añadirá dos atributos nuevos:

- **Material* mat**: Puntero a un material, que se usará para renderizar esta malla

- **std::vector<glm::uint32>* vTriangleIdxList**: Lista de identificadores de vértices, cada triplete de identificadores serán un triángulo.

Se añadirán las nuevas funciones:

- **Getter/Setter** para las nuevas variables
- **addTriangle(glm::uint32 vld1, glm::uint32 vld2, glm::uint32 vld3)** : Método para añadir un nuevo triángulo a la malla, usando tres identificadores de vértices

Clase GL4Render

Extiende la clase GL1Render, aprovechando los constructores, inicializadores y métodos comunes. Esta clase reimplementará los siguientes métodos:

- **virtual void setupObject(Object* obj)** : Creará los buffer objects necesarios para poder almacenar los datos de vértices en GPU. Se aconseja guardar los buffer objects usando un std::map que los almacene usando los identificadores de malla como clave.
- **virtual void removeObject(Object* obj)** : Liberará los buffer objects de un objeto 3D inicializados anteriormente
- **virtual void drawObjects(std::vector<Object*>* objs)**: Limpiará buffers de color y profundidad, seteará la matriz modelo del objeto seleccionado, por cada objeto llamará a la función de draw object de GL4, y por último actualizará buffers de pantalla.
- **virtual void drawObject(Object* obj)**: Dibujará un objeto utilizando GL4, accediendo a los materiales y shaders inicializados anteriormente para este objeto.

Se proporciona la siguiente estructura para almacenar los identificadores de buffer Objects:

```
typedef struct boIDS_t
{
    unsigned int id;
    unsigned int vbo;
    unsigned int idxbo;
}boIDS_t;
```

Modificaciones clase FactoryEngine

Añadir un nuevo backend gráfico llamado “GL4”. Si está seleccionado, al invocar “**getNewRender**” devolverá una clase de tipo GL4Render (el resto de métodos permanecen iguales tanto si es GL1 como si es GL1).

Añadir un nuevo método “**getNewMaterial**”. Devolverá un material de tipo GLSLMaterial en caso de estar seleccionado GL4 como backend.

Modificaciones clase System

Para facilitar el acceso a los datos de los objetos que se están renderizando en un momento dado, se aconseja añadir el siguiente atributo estático:

- **static glm::mat4 ModelMatrix**: Contiene la matriz modelo del objeto que se está renderizando actualmente
- Añadir **Getter/setters** para acceder al atributo ModelMatrix desde el método “prepare” del GLSLMaterial.

Modificaciones clase TrianguloRot

El constructor de “TrianguloRot” debe crear una malla con un material que contenga los shaders para poder dibujarlo. Además de tener los vértices de un triángulo, debe pedir un material a FactoryEngine y setearlo en su malla. En ese material, debe cargar los programas de vértices y fragmentos proporcionados a continuación:

Vertex Shader:

```
#version 330
uniform mat4 mMat;
in vec4 vPos;

void main()
{
    gl_Position=mMat*vPos;
}
```

Fragment Shader:

```
#version 330
out vec4 fragColor;
void main()
{
    fragColor=vec4(1.0f,1.0f,1.0f,0.0f);
}
```

Método “main”

El programa principal realizará las siguientes tareas:

- Inicializar el backend de la clase “FactoryEngine” para poder usar OpenGL 4.0 y el input manager por defecto.
- Inicializar la clase System
- Crear un objeto de tipo “TrianguloRot” en la posición <0,0,0>, y añadirlo al sistema
- Ejecutar el mainLoop del sistema, hasta que el usuario presione la tecla “e”